

Web Development Assignment

Semester 1, 2026

DUE ON: 11:59 PM, Thursday 28 May 2026

(Progress Report to be submitted in Week 7 or Week 8 lab session)

Assignment Worth: 50% of total marks

NB:

- Assignments will be accepted up to five (5) days late, but a penalty of 5% per day (or part of a day) late will be imposed depending on the circumstances.
- **This is an individual assignment.** Students are referred to the school's policy on plagiarism. A confirmed case will incur zero mark to all the involved students.
- **This Assignment has three parts:**
 1. Part 1 must be deployed on the webdev server (60%)
 2. Part 2 can be deployed anywhere with browser access (35%)
 3. Part 3 is a video demo to both Part 1 and Part 2 (5%)
- **Submission:** Combine all three parts into a single Assign.zip and upload on Canvas.
- **Progress Report:** A one-page progress report containing detailed plan, progress, and a snapshot of the application should be submitted in person to your TA in Week 7 or Week 8 lab. The report is counted as lab marks.

Part 1 (60% of the whole assignment)

1. Overview

The aim of Part 1 of the assignment is to develop an understanding of building web applications using Web 2.0 techniques, PHP, and MySQL. You will need to create all the appropriate HTML, JavaScript, PHP files and database tables. You should save and test all your HTML and PHP files on *webdev.aut.ac.nz* server in the directory "*htdocs/assign*".

2. Tasks

Part 1 is to develop a simple web-based taxi booking system called **CabsOnline**. It allows passengers to book taxi services from any of their internet connected computers or mobile phones. The techniques you are going to use include the Web 2.0 techniques (JavaScript/HTML, JS Await-fetch, CSS, and DOM), MySQL and PHP.

Two webpages/functions (booking and admin) of such an online service that must be completed for this assignment are specified in the following two sub-sections. Other components such as map-based interaction, querying service for drivers, monitoring services for customers, payment processing, detailed processing for assigning taxi could be implemented in Part 2 of the assignment.

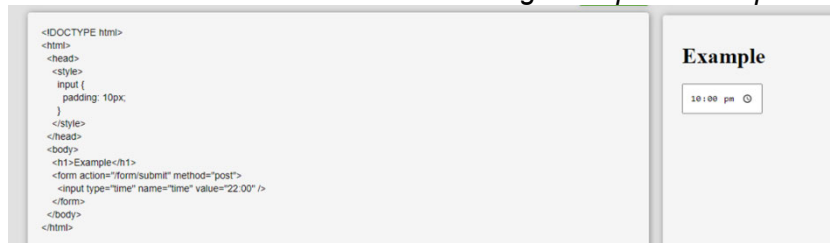
2.1 The Booking Function

NB: For this task, you must use the same name "booking" for all types of files for this task. For example, booking.html, booking.php, booking.js.

This function is used to allow a passenger to submit a taxi booking request in Auckland and its surrounding areas.

- 1) Design and create a MySQL table for storing information of all requests. For each request received, information including the generated booking number, customer's inputs, the generated booking date and time, and the generated assignment status must be stored in this table.
- 2) On the client side, design an HTML page (booking.html) and a client-side program to take inputs for a booking request, and pass them to server for processing using asynchronous communication (fetch). You need to validate these inputs before sending them to the server:
 - **Customer Name** (Text input type. Must set its name attribute to "cname": `<input type="text" name="cname">`. NOT NULL)
 - **Phone Number** (Text input type. Must set its name attribute to "phone": `<input type="text" name="phone">`. NOT NULL). Phone number must be all numbers with length between 10-12
 - **Unit Number** (Text input type. Must set its name attribute to "unumber": `<input type="text" name="unumber">`. Optional)
 - **Street Number** (Text input type. Must set its name attribute to "snumber": `<input type="text" name="snumber">`. Not NULL)
 - **Street Name** (Text input type. Must set its name attribute to "sname": `<input type="text" name="sname">`. NOT NULL)
 - **Suburb** (Text input type. Must set its name attribute to "sbnname": `<input type="text" name="sbnname">`. Optional)
 - **Destination Suburb** (Text input type. Must set its name attribute to "dsbnname": `<input type="text" name="dsbnname">`. Optional)¹
 - **Pick-Up Date** (text or date input type. Must set its name attribute "date". NOT NULL, should initially contain the current date in dd/mm/yyyy format e.g. 01/05/2021. This can be edited by the user.
 - **Pick-up Time** (text or time input type. Must set its name attribute "time". NOT NULL, should initially contain the current time in 24h format HH:MM format e.g. 18:30. This can be edited by the user. **Please note that the pick-up date and time must not be earlier than current date and time on the client machine.**

*NB: The value attribute must be set to the 24-hour format. However, the time input type display format is either in 12 or 24-hour **depending on OS locale** when viewing in Chrome. Therefore, the browser display could either be in 12 or 24-hour format. The following example is acceptable.*



- 3) On the server side, for each booking request, a server-side program is required to generate a unique and incremental booking reference number, booking date and

¹ Note that it is not necessary to validate the genuineness of an address. If you do want to make the application more realistic, you can use `google.maps.Geocoder()` Web API. Please refer to Google Map API online document or <https://codepen.io/pavenuto/pen/dXkgEz> on how to do that.

time and a status with initial value “unassigned”, and add them together with customer’s inputs into the MySQL table. The booking reference number must follow the format of “BRN00001”, that is: the first three characters are upper case “BRN”, then the rest five character are digits.

- 4) On the client side, the client-side program needs to display a returned confirmation message upon receiving the returned information from the server (as we are using Web 2.0, the returned message should be inserted on the same booking.html page). The confirmation message must follow the following format requirements:

- Confirmation message must include keywords of “booking reference number”, “pickup time” and “pickup date”.
- The booking reference number must follow the correct format (as stated in Task 2.1.3)
- The pickup time must be displayed in the 24-hour format
- The pickup date must be displayed in the DD/MM/YYYY format

The confirmation message should be similar to the example below:

Thank you for your booking!	
Booking reference number:	BRN00001
Pickup time:	16:01
Pickup date:	25/1/2026

You must make sure the message is inside a <div> element of the booking.html file. You may choose to use an appropriate tag for the message, but the id attribute must be set to “reference”. For example, you will need <p id=“reference”> if you choose to put the confirmation message in a <p> element.

2.2 The Admin Function

NB: You must use the same name “admin” for all types of files for this task. For example, admin.html, admin.php, admin.js.

This component allows administrative people of CabsOnline to view those taxi booking requests that need to be assigned as soon as possible and to assign taxi for a particular booking request. Note that authentication is not required though it is necessary in the real application. The specific functions of this component include:

- 1) Design an HTML page (admin.html) that handles booking request searches. It should be realized by a single button to display pickup request information. This booking request search consists of two parts:
 - **A text field for booking request searching** (Text input type. Must set its name attribute to “bsearch”: <input type=“text” name=“bsearch”>. Optional). User can either input a reference number or nothing:
 - i. If user input is non-empty, it must be verified by checking the format of the reference number. When the input is not in the valid format, an error message is displayed accordingly.
 - ii. After verifying the format of the reference number, the booking record associated with this reference number is returned by the server.

- iii. If user input is empty, then a list of bookings with a pickup time **within 2 hours from the current time** is returned by the server.

Task 2.2.3 provides more details.

- **A Button to submit the search request** (button input type. Must set its name attribute to "sbutton": `<input type="button" name="sbutton">`). Once the button search booking is pressed, a client-side program issues a query request to the server (using Fetch);
- 2) On the server side, a server-side program responds to the search request by executing a query on the MySQL database to find the matching records.
- For input with a reference number, the record exactly matches the reference number should be returned or no record is returned if there is no match;
 - For an empty input, the server-side program should execute a query on the database to find those "unassigned" booking requests with a pickup time within 2 hours from the current time only.
 - Information returned to the client should include the booking reference number, customer name, contact phone number, pickup suburb, destination suburb, and pickup date and time.
- 3) When getting a search request, the returned information must be displayed in a table inside a `<div class="content">` container in the admin.html file.

The following table header fields must be used:

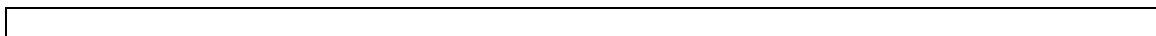
- Booking reference number
- Customer name
- Phone
- Pickup suburb
- Destination suburb
- Pickup date and time
- Status
- Assign

Each table row must contain one record. For each row, the "assignment" field must contain a button with name "Assign". When the "assign" button is clicked, the status of this record (booking request) needs to be updated on both the database and the HTML page.

Below is a sample screenshot of the table:

Booking Reference Number	Customer Name	Phone	Pickup Suburb	Destination Suburb	Pickup Date and Time	Status	Assign
BRN00001	John Smith	123-12345678	Auckland CBD	Northcote	01/05/2025 13:15	Unassigned	Assign
...	...	...	...	...	...	...	...

- 4) In the same server-side program, it should also respond to taxi assignment requests. The booking reference number is sent to the server side when an 'Assign' button is clicked in admin.html. Upon receiving the client-side request, the server-side program should complete an update to the MySQL database to change the status from "unassigned" to "assigned" for the matching record. After that, a confirmation message should be returned and inserted into admin.html. The message must contain the booking reference number. You might optionally disable (gray-out) the corresponding 'Assign' button.
- Below is a sample screenshot after the assignment:



Congratulations! Booking request BRN00001 has been assigned!							
Booking Reference Number	Customer Name	Phone	Pickup Suburb	Destination Suburb	Pickup Date and Time	Status	Assign
BRN00001	John Smith	123-12345678	Auckland CBD	Northcote	01/05/2025 13:15	Assigned	Assign
...	...	...	...	...	...	...	...

2.3 The AI Usage Declaration Form

Fill out the “AI Usage Declaration.doc” form for Part 1.

Preparation for Submission

You should ensure that all files used for the assignment sit in a directory called “*assign*” within your server account. The directory should contain no sub-directories (i.e., all files are placed directly under the “*assign*” directory).

Please follow the following steps:

- Make sure your code works on the webdev server in the folder “assign”.
- Download the folder “assign” (including all the files in it) to your local machine.
- Zip this folder and rename it as `Part1_<your student username>.zip`.
This file will be combined with Part 2 work for submission.

The files should include:

- two HTML files: booking.html and admin.html;
- any JavaScript files that you created
- any CSS files that you created
- any PHP files that you created
- a text file “mysqlcommand.txt” that includes the MySQL commands that you used to create the database
- a text file readme.txt that includes
 - a list of all the files in the system;
 - brief instructions on how to use the system.
- The AI Usage Declaration Form.

For each submitted file, we require clear comments including student information, description of the file, and description of each function defined in this file.

The MySQL database that you use should be constructed in your *webdev* account. After submission, you are not allowed to change any of the submitted files in the *assign2* directory on your *webdev* account; time stamps will be checked.

Part 1 Marking Scheme (60%)

NB: If your work cannot run on the webdev server, you will get 0 mark for this part.

Task	Assessment item	Received	Marks
2.1.1	Design and create a MySQL table		2

2.1.2	• Design an HTML page (booking.html) (2) • A client-side program to take inputs for a booking request and pass them to server for processing using Fetch or XHR. (2) • All required input items must be provided except the optional ones. (2) • The pick-up date and time must be no earlier than current date and time (2).		8
2.1.3	• Generate a unique and incremental booking reference number, booking date and time and assignment status (4) • Insert into the MySQL table (4)		8
2.1.4	• Confirmation message displays the following information: ○ Booking reference number (2) ○ Pickup time (2) ○ Pickup date (2) • Information displayed in the message follows the correct format (2)		8
2.2 Initial landing page	On the landing page (admin.html), the request searching functionality is realised. The following can be found on the page: • A text input field for searching. Name attribute is set to "bsearch". (2) • A search button where the name attribute is set to "sbutton". (2)		4
2.2 Booking Request Search & Taxi Assignment	When there is a non-empty input: • The input is verified by checking the format of the reference number. If the input is not in a valid format, an error message is displayed accordingly. (2) • A matched record is displayed on the page when the provided reference number already exists in the database. It should be presented in a table including the following information: (8 in total) ○ booking reference number ○ customer name ○ phone number ○ pickup suburb ○ destination suburb ○ pickup date and time ○ status • An "assign" button should be present at the end of each table row. (2) • When clicking on the "assign" button, the record should change its status from "unassigned" to "assigned".(2)		18

	- Once the taxi assignment is completed, a confirmation message is displayed. (2) When there is an empty input: - A list of bookings with pickup times within 2 hours from the current time is displayed on the page. (2)		
Overall quality	Comments are written in all files. For each file, the comments include descriptions of the file and all the functions used. Student information is included at the top of the files. Files should not include code that is commented out. The readme.txt file includes a list of all the files as well as brief instructions on how to use the system. Code quality. Evidence may include: - Consistent naming conventions for variables, methods and functions. - Indentation is used appropriately for good readability - Structured comments are presented throughout the files Usability of user interface - No redundant or misleading information can be found in the system. All information presented are consistent. - Buttons all function as intended. The actual functionalities match the text of the buttons.		10
	AI Usage Declaration Form		2
	Total		60

Part 2 (35% of the whole assignment)

1. Overview

The aim of Part 2 of the assignment is to **use AI** (e.g., GitHub Copilot, Cursor, Claude Code, Stackblitz Bolt, or any general-purpose AI such as ChatGPT) to refactor and extend your Part 1 work. You may either reuse the existing PHP code or use another programming language to rewrite the existing functions. This new application **extends**

the functionality using a modern web framework such as React. You are allowed to use any programming languages or frameworks, and you are allowed to deploy the application anywhere accessible by a browser (e.g., GitHub Pages, Stackblitz, or AUT webdev server).

NB: If you choose to do this part on webdev server, use a separate directory "part2". (Part 1 code should not be changed)

2. Tasks

2.1 Extension using Modern Frameworks

Extend the functionality of Part 1 (up to four functions). Potential functions include:

- map-based interaction
- querying service for drivers
- monitoring services for customers
- payment processing

You can also come up with your own idea.

This task will be assessed based on the richness and usability of the work.

2.2 Documentation

Write up a document README.DOC with the following sections:

1. Public URL of the deployed application
2. Technology stack used (React version, libraries, map provider)
3. How to run and build the project locally
4. Microservice API endpoints used (local and remote)
5. Feature descriptions
6. Testing instructions
 - Example booking references
 - Sample driver IDs
7. Any limitations or known issues
8. Reflection on AI-supported development process

Preparation for Submission

Put the following files into a single `Part2_<your student username>.zip`.

Your ZIP must include:

1. Web Application Files including all React (or equivalent) source files (e.g., /src, /public, package.json, build config)
2. Build output files (e.g., /dist or /build)
3. Any additional JavaScript, CSS, or helper modules you've created
4. README.DOC

Part 2 Marking Scheme (35%)

NB: If your work is not accessible or cannot run, you will get 0 mark for this part.

Task	Assessment item	Received	Marks
2.2	Richness of functionality with four moderate-size functions (10) Clean, modern UI with responsive design (10) Robustness including proper error handling (3)		23
2.3	README.doc will be assessed based on the clarity, completeness, level of detail, and logical flow demonstrated in each section Clarity (3) Writing is easy to understand and free of ambiguity. Sentences are concise, well-structured, and logically connected. Technical terms are correctly used and explained when necessary. Completeness (3) All required sections are present and thoroughly addressed. No essential information is missing or left implied. The document provides enough context for readers to fully understand the project without external clarification. 3. Level of Detail (3) Explanations include appropriate depth and specificity. Important design decisions, processes, and justifications are clearly discussed. The document avoids being overly brief or unnecessarily verbose. 4. Logical Flow (3) Ideas progress in a coherent, well-organized manner. Sections transition smoothly and follow a meaningful structure. The document is arranged in a way that supports comprehension and reflects professional documentation standards.		12

Part 3 Video Demo (5% of the whole assignment)

Create a 5-7 minute video to demonstrate your work for both Part 1 and Part 2. This video should include the following sections:

1. System demo including all the functionalities with example user input and proper responses of the system.
2. Introduce the code structure and technology used.

Video format: MP4, MOV

Useful link: [How to Record a Presentation with Microsoft Teams?](#)

Part 3 Marking Scheme

Video demo will be assessed based on clarity, completeness, level of details, and logic of flow

Task	Assessment item	Received	Marks
1	System demo		3
2	Code structure and technology used		2